

COMPILADORES

PROJETO – Fase 1 – Análise Léxica e Análise Sintática

OBJETIVO

A proposta desta atividade é a implementação das duas primeiras etapas do *front-end* do projeto de uma linguagem de programação, ou seja, as fases de análises léxica e sintática de um compilador.

Na implementação do compilador o analisador léxico deve atender as demandas do analisador sintático. A interação entre o analisador léxico e o analisador sintático se dá por meio da função *consume()* (analisador sintático) que realizará chamadas à função *obter_atomo()* (analisador léxico).

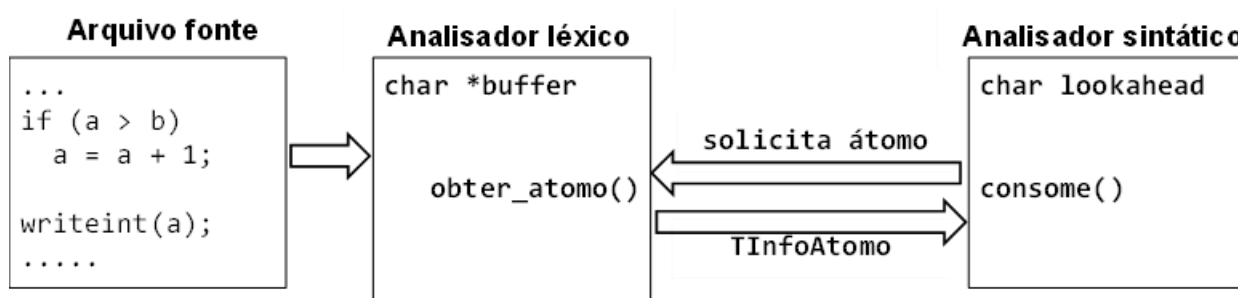


Figura 1: Interação entre Analisador Léxico e Sintático

LINGUAGEM DE PROGRAMAÇÃO - MiniPascal

Você construirá as etapas de análise léxica e análise sintática da linguagem MiniPascal (que se trata de uma simplificação da linguagem Pascal) e deve utilizar a linguagem C para a implementação.

MiniPascal é um subconjunto da linguagem Pascal. Como é um subconjunto, todo programa MiniPascal é um programa Pascal válido que pode ser executado.

MiniPascal restringe a linguagem Pascal para ter apenas inteiros e booleanos. Os comandos permitidos são aqueles que podem ser usados com os tipos de dados mencionados. Os literais não são considerados para manipulação, apenas nos prints para mensagem de interação com o usuário (em Pascal o literal é colocado entre aspas duplas).

MiniPascal é uma linguagem case sensitive.

EXEMPLO DE CÓDIGO CORRETO

```
program correto ;
var a, b, c : integer ;
var d, e, f : boolean ;

procedure proc(var a1 : integer ) ;
var a, b, c : integer ;
var d, e, f : boolean ;
begin
```



```
a := 1 ;
if ( a < 1 ) then
    a := 12 ;
end ;

begin
    a := 2 ;
    b := 10 ;
    c := 11 ;
    a := b + c ;
    d := true ;
    e := false ;
    f := true ;
    write ( b ) ;
    if ( d ) then
        begin
            a := 20 ;
            b := 10 * c ;
            c := a div b ;
        end ;
    while ( a > 1 ) do
        begin
            if ( b > 10 ) then
                b := 2 ;
            a := a - 1 ;
        end ;
    end .
```

ESPECIFICAÇÃO LÉXICA

Um programa MiniPascal deverá estar codificado em um único arquivo fonte, sem fazer referências a detalhes externos a ele.

A seguir são apresentadas a gramática da linguagem MiniPascal, que deve ser seguida rigorosamente, e as especificações léxicas da linguagem, onde são definidos os átomos da linguagem.

Comentários

Aparecem delimitados por “/*” e “*/”. Tudo que segue os símbolos “/*” é ignorado pelo MiniPascal até que se encontre “*/”.

```
/* comentário */
```

Identificadores (variáveis)

Podem ser criados com caracteres alfanuméricos e o caractere ‘_’. Um identificador não pode começar com um número, ou seja, é iniciado por uma letra ou ‘_’ e é seguido de zero ou mais letras, dígitos ou ‘_’



Value
company_number

12Val /* inválido */
Ex\$ /* inválido */
company-name /* inválido */

Tipos de dados

Numerais: Inteiros (tipo integer)

Booleano: False | True (tipo boolean)

Operadores

+	-	div	*	and	or	not
<	>	<>	<=	>=	:=	

Delimitadores

() , : . ;

Palavras Reservadas

program	begin	end	procedure
if	then	else	while
do	and	or	not
var	integer	boolean	true
false	write		

SINTAXE

1. <programa> ::=
 program <identificador> ; <bloco>.
2. <bloco> ::=
 [<parte de declarações de variáveis>]
 [<parte de declarações de sub-rotinas>]
 <comando composto>

Declarações

3. <parte de declarações de variáveis> ::=
 var <declaração de variáveis>
 {; <declaração de variáveis>;}
4. <declaração de variáveis> ::=
 <lista de identificadores> : <tipo>
5. <lista de identificadores> ::=



<identificador> {, <identificador>}

6. <parte de declarações de subrotinas> ::=
 {declaração de procedimento> ;}

7. <declaração de procedimento> ::=
 procedure <identificador> [<parâmetros formais>] ; <bloco>

8. <parâmetros formais> ::=
 (<seção de parâmetros formais> { ; <seção de parâmetros formais>})

9. <seção de parâmetros formais> ::=
 [**var**] <lista de identificadores> : <tipo>

10. <tipo> ::=
 integer | **boolean**

OBS: Todos os identificadores devem ser declarados antes de serem utilizados.

Comandos

11. <comando composto> ::=
 begin <comando> { ; <comando>} **end**

12. <comando> ::=
 <atribuição>
 | <chamada de procedimento>
 | <comando composto>
 | <comando condicional 1>
 | <comando repetitivo 1>
 | **write**(<identificador>)

13. <atribuição> ::=
 <variável> := <expressão>

14. <chamada de procedimento> ::=
 <identificador> [(<lista de parâmetros>)]

15. <lista de parâmetros> ::=
 [(<identificador> | <numero> | <bool>)
 {, (<identificador> | <numero> | <bool>) }]

16. <comando condicional 1> ::=
 if (<expressão>) **then** <comando> [**else** <comando>]

17. <comando repetitivo 1> ::=
 while (<expressão>) **do** <comando>



Expressões

18. `<expressão> ::=`
 `<expressão simples> [<relação> <expressão simples>]`
19. `<relação> ::=`
 `= | <> | < | <= | >= | >`
20. `<expressão simples> ::=`
 `[+ | -] <termo> {(+ | -) <termo>}`
21. `<termo> ::=`
 `<fator> {( * | div) <fator> }`
22. `<fator> ::=`
 `<variavel>`
 `| <número>`
 `| <bool>`
 `| ( <expressão simples> )`
23. `<variável> ::=`
 `<identificador>`

Números e Identificadores

24. `<bool> ::=`
 `true | false`
25. `<numero> ::=`
 `<digito> {<digito>}`
26. `<digito> ::=`
 `0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9`
27. `<identificador> ::=`
 `<letra> {<letra> | <dígito>}`
28. `<letra> ::=`
 `_ | a | b | ... | z | A | B | ... | Z`

EBNF:

`{α}` = repetição da cadeia a zero ou mais vezes
`α | β` = `α` ou `β` devem ser escolhidos
Não terminais aparecem entre `<` e `>`.



ENTREGAS

Etapas:

- (1) Gramática livre de contexto e autômatos finitos
- (2) Analisador léxico
- (3) Analisador sintático

OBS₁: cada entrega deve ser composta por 2 arquivos: código e documentação (ou simplesmente código muito bem comentado).

OBS₂: a não entrega de uma etapa zera o projeto.

ETAPA#1 – Expressões Regulares, Gramática Livre de Contexto e Autômatos Finitos

- A partir das definições da linguagem específica e da sugestão de sintaxe, apresentar as expressões regulares e construir a gramática livre de contexto correspondente.
- Construir os autômatos para cada classe de elementos do vocabulário terminal da linguagem: letras, dígitos, números, identificadores, operadores, palavras reservadas, pontuação, comentários.
- Testar cada classe gerada.
- Juntar todos os autômatos em um único para que represente todos os elementos.

OBS₃: encaminhar o autômato completo e não separadamente para cada padrão definido.

#dica - utilizar a ferramenta JFLAP (www.jflap.org) para a construção dos autômatos.

ETAPA#2 – Analisador léxico

- Desenvolver um programa que realize a análise léxica.
- A partir de um arquivo contendo um código (programa) em MiniPascal, o analisador léxico deverá produzir um arquivo com os tokens identificados e seus valores correspondentes (se necessário) e apresentar na tela o mesmo resultado.
- Considerar que nem todos os tokens estão separados por um espaço.
- Cada linha do arquivo de saída e do print na tela deve conter o número da linha do arquivo fonte onde o token aparece e especificar o token reconhecido.

```
Número da Linha do Átomo # NomeToken | Atributo  
Por exemplo: 11# IDENTIFICADOR | var1  
11# ID | var1
```

- Cada token deve ser armazenado em um registro (*struct*) com um conjunto de campos de tipos diferentes para armazenar cada uma das informações referente ao token.
- Definir as constantes inteiras como um tipo enumerado.

```
typedef enum{  
    ERRO,  
    IDENTIFICADOR,  
    NUMERO,
```



EOS

```
}TAtomo;
```

- Fazer uso do que foi implementado na Etapa#1.
- O analisador léxico deve ser capaz de lidar com os erros léxicos encontrados no programa. A mensagem de erro, “ERRO LÉXICO” juntamente com a sequência lexicamente errada, devem ser apresentadas e todo o processo finalizado.
- Utilizar os nomes de módulos sugeridos no item OBJETIVO.

ETAPA#3 – Analisador sintático

- Construir um programa que realize a análise sintática.
- A partir da estrutura com os tokens gerados pelo analisador léxico (Etapa#2), realizar a análise ascendente ou descendente com base na gramática da linguagem.
- O analisador sintático deve ser capaz de lidar com erros sintáticos encontrados no programa. A mensagem de erro, “ERRO SINTÁTICO” juntamente com o token incorreto e a linha do código fonte correspondente, devem ser apresentados e todo o processo finalizado.
- É necessário entregar os analisadores léxico e sintático funcionando em conjunto.
- Utilizar os nomes de módulos sugeridos no item OBJETIVO.

CRITÉRIO DE AVALIAÇÃO

- Etapa#1 – 20%
- Etapa#2 – 40%
- Etapa#3 – 40%

Funcionamento do programa:

- Caso o programa não compile ou não execute será atribuída a nota 0 ao trabalho.
- Caso o programa apresente *warning* durante a compilação ou não finalize com retorno igual a 0, será descontado 1.0 (um ponto) por *warning* relatado.
- O trabalho deve ser desenvolvido na linguagem C e será testado usando o compilador do MinGW com VSCode.
- Compile seu programa com o seguinte comando abaixo, considere que o programa fonte do seu compilador seja compilador.c:

```
gcc -Wall -Wno-unused-result -g -Og compilador.c -o compilador
```

Atendimento à especificação do enunciado:

O seu programa deve seguir a gramática definida e realizar a leitura de programa fonte armazenado em arquivo com o nome informado por linha de comando.

Clareza e organização, programas com código confuso (linhas longas, variáveis com nomes não-significativos,) e desorganizado (sem indentação, sem comentários,) também serão penalizados.

Entregar um arquivo Readme.txt explicando até a parte do trabalho que foi concluído, além de relatar quaisquer bugs ou erros identificados na sua implementação. No arquivo você pode compartilhar alguma decisão de *design* e implementação que foi tomada durante o desenvolvimento.



PRAZO FINAL

Entrega: 30/09

GRUPOS

O projeto pode ser desenvolvido individualmente ou em grupo de até 2 alunos. É imprescindível que os nomes dos integrantes do grupo sejam mencionados no início do arquivo fonte do trabalho.

LEMBRETE

Fica terminantemente proibido o uso de ferramentas de Inteligência Artificial, como o ChatGPT, para a geração automática do código do projeto. Qualquer tentativa de burlar esta restrição será considerada uma infração disciplinar, conforme o COMUNICADO DA FCI de 05/02/2025.